Module 1: Lesson 3

Introduction to Machine Learning

Outline

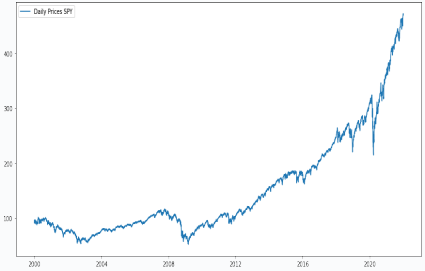
- Classification models and scaling
- Logistic Regression
- Performance of classification models

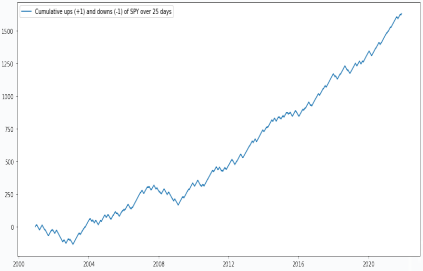
Classification models

Classification problems represent the most frequent tasks encountered in supervised ML applications. We want to predict whether an instance belongs to one class among an array of classes.

We can easily transform our return-prediction model developed in Lessons 1 and 2 into a classification problem.

That is, we may be interested in just predicting whether the stock market goes up or down, rather than a precise return. We can design simple short/long strategies based on our predictive model.





Scaling of input features

Feature scaling is one of the most important tasks to perform before we train an ML model, and this is not specific to classification problems.

Machine learning algorithms perform badly when the input features have very different scales, which tends to slow down and worsen the performance of the optimization algorithms (e.g., gradient descent).

Min-max scaling (or normalization): Values are shifted and re-scaled so that they end up ranging from 0 to 1:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - min_j}{max_j - min_j}$$

Standardization: subtracts the mean value and divides by the variance so that the resulting distribution has zero mean and unit variance. It does not constrain values, which may be a problem in some algorithms.

Whitening: Using unsupervised ML methods, we create new de-correlated features, each of which is scaled to unit variance.

- **Important**: We first only scale the training set (using its min, max, mean...), which can then be applied to the validation and test sets.

Logistic Regression: Main elements

Logistic Regression is a popular approach in ML to deal with binary classification problems.

The model estimates the probability that an instance belongs to a particular class: For instance, what is the probability that the market will go up?

- In our training sample, we have instances with labels y that denote if the market has gone up $y = 1$ or gone down $y = 0$ and some input features X.

If the estimated probability is above 50%, the model predicts that the instance belongs to the positive class, $\widehat{y} = 1$; otherwise, it belongs to the negative class, $\widehat{y} = 0$.

To obtain the estimated probabilities for an instance i, the Logistic Regression model computes a score as a weighted sum of the input features (plus a bias term) and then outputs the logistic of this result:

$$\widehat{p}^{(i)} = \frac{1}{1 + exp\left(-x^{(i)}\theta\right)}$$

The function $\sigma(z) = \frac{1}{1+exp(-z)}$ is known as the sigmoid function. If we use the 50% threshold to determine the predicted class, the logistic regression then predicts 1 if $x^{(i)}\theta$ is positive and 0 otherwise.

Logistic Regression: Training

We train the parameter vector θ so that the model estimates high probabilities for positive instances ($y^{(i)} = 1$) and low probabilities for negative instances ($y^{(i)} = 0$).

This idea is captured by the following cost function:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} log\left(\widehat{p}^{(i)}\right) + \left(1 - y^{(i)}\right) log\left(1 - \widehat{p}^{(i)}\right)\right]$$

We have no closed form for the vector θ that maximizes the cost function above, but gradient descent is guaranteed to find the global minimum as long as the learning rate is small enough and we wait for enough time.

We can obtain the gradient from the derivatives from the cost function:

$$\frac{\partial}{\partial\theta_j}J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} - \sigma\left(x^{(i)}\theta\right)\right]x_j^{(i)}$$

We can regularize θ using the Ridge and Lasso penalties we described in Lesson 2.

Softmax Regression

We can generalize the binary classifier of the Logistic Regression model to multiple classes: This is called *Softmax Regression* or *Multinomial Logit Regression*.

The Softmax Regression model computes a score for each instance i and category c, $x^{(i)}\theta^{(c)}$, and estimates the probability of each class by using the softmax function.

- Notice that each category has a specific vector of parameters, $\theta^{(c)}$, with dimension equal to the number of input features, plus a bias term.

With C different categories, the estimated probability of category c for instance i is:

$$\widehat{p}_c^{(i)} = Softmax_c\left(x^{(i)}, \Theta\right) = \frac{exp\left(x^{(i)}\theta^{(c)}\right)}{\sum_{j=1}^{C} exp\left(x^{(i)}\theta^{(j)}\right)}$$

where Θ collects all the parameters of the model. The model prediction $\widehat{y}^{(i)}$ will be the category with the highest score $x^{(i)}\theta^{(c)}$.

Performance measures in classification

Evaluating a classifier is often significantly trickier than evaluating a regression model.

Accuracy: The frequency with which the classifier is successful at classifying the labels in the test sample.

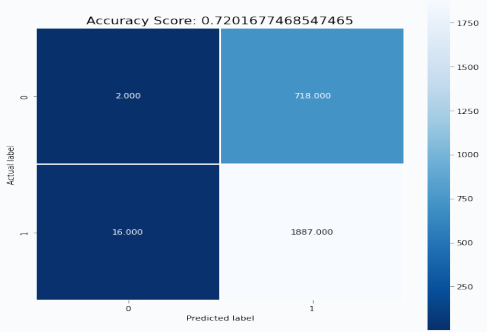
- Even a terrible classifier may be accurate. If the baseline probability of the positive class is 5%, a classifier that always predicts a negative class will mechanically achieve a 95% accuracy.

Using a Logistic Regression model, in our stock market prediction task, we find a 72% success rate in predicting whether the stock market is going up or down.

- The predictive power arises simply from the model predicting most of the time that the stock market is going up, which happens with roughly 72% probability.

Because accuracy is a somewhat misleading performance measure, we will describe other, more useful, metrics next.

Confusion Matrix



Confusion Matrix: After obtaining a set of predictions, we construct a matrix where each row represents the actual class of the instances, while each column represents the predicted classes. Each cell in the matrix then counts the number of times instances of class A are classified as B.

A perfect classifier would have nonzero values only on its main diagonal (top left to bottom right).

In the binary classifier case, we can represent the confusion matrix as:

$$Confusion = \begin{pmatrix} True\ Negatives & False\ Positives \\ False\ Negatives & True\ Positives \end{pmatrix}$$

Precision and recall

Precision: captures the proportion of accurate predictions over all the 1s that have been predicted by the model.

$$Precision = \frac{TP}{TP + FP}$$

where TP is the total number of true positives and FP is the number of false positives.

Recall: captures the proportion of accurate predictions over all the 1s that appear in the sample.

$$Recall = \frac{TP}{TP + FN}$$

Precision and recall are usually combined in the F_1 score $= 2\frac{Precision \times Recall}{Precision + Recall}$

The F_1 score favors classifiers with similar precision and recall. Sometimes, we do not want that:

- If we develop a model to detect the probability of a large drop (crash) in the stock market, in which case the category is 1, we may be willing to sacrifice some accuracy, predict the crash when it does not take place, against some recall, do not predict the crash when the crash actually happens.

ROC & AUC

We cannot have, simultaneously, classifiers with high precision and high recall.

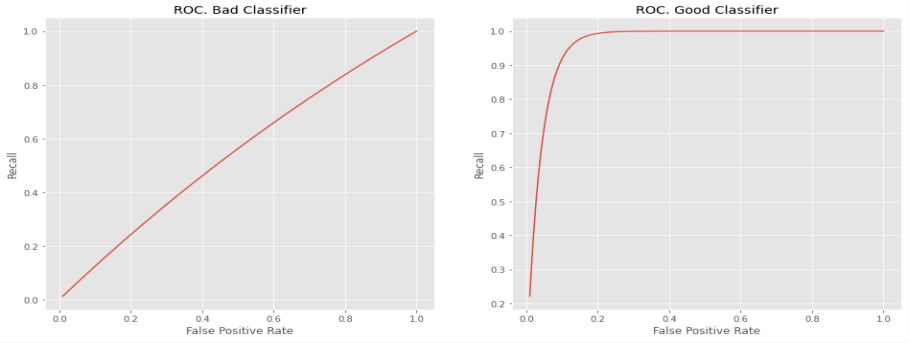
- To determine which class the model is predicting, we set a threshold value on the predicted probabilities to distinguish between a positive and a negative class, such as 0.5 in the logistic regression.
- If a classifier sets a high threshold to increase precision, it is going to increase the number of positive cases that are going to be predicted as negative, reducing recall.
- Otherwise, if a classifier sets a high bar to increase recall, it is going to be at the cost of predicting more negative cases as positives, reducing precision.

Still, a good classifier can increase its precision without a large decrease in recall, and vice versa.

To show this, we plot the Receiver Operating Characteristic (ROC) curve and compute the Area Under the Curve (AUC).

- The ROC curve plots the recall of the classifier against its rate of false positives for different thresholds for the estimated probabilities.
- A good classifier has a steep ROC that reaches a Recall close to 1 without generating a high rate of false positives. In that case, the AUC is close to 1.

ROC & AUC: Illustration



Summary of Lesson 3

In Lesson 3, we have looked at:

- Classification models in Supervised ML
- The Logistic Regression Model
- Performance measures for ML classification models

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.* 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to obtain further insights on the workings of ML classification tasks and the performance of a stock market predictor.

In the next lesson, we will develop an application of ML classification models for a credit scoring problem.